

Project ESCAPE: Entropy & Semantic Compute Allocation in Programming Environments — Structural Discovery via Entropy in the Byte Latent Transformer (BLT) Across Code and Natural Language

Priyanka Agarwal
2024101016

Shashwat Mukadam
2024102008

Divyansh Atri
2024113001

Team Trimax

CS7.501 Advanced NLP – Interim Project
Proposal

Abstract

Traditional Large Language Models rely on tokenization heuristics (e.g., BPE) that statically compress text, allocating equal compute to both predictable boilerplate and highly complex logic. The recently introduced Byte Latent Transformer (BLT) eliminates this paradigm by operating on raw bytes and dynamically grouping them into patches based on next-byte prediction entropy. Project ESCAPE investigates the interpretability and robustness of BLT’s entropy-guided patching mechanism. Specifically, we evaluate whether BLT’s dynamic compute allocation naturally discovers and aligns with the rigid structural syntax of programming languages (Abstract Syntax Trees) compared to the fluid entropy distribution of natural language prose. We further expand this analysis to investigate the model’s robustness against character-level adversarial noise and its capacity to dynamically allocate higher compute to semantically complex, memory-unsafe operations.

1 Introduction and Motivation

Tokenization has long been the standard for training sequence models, but it enforces a static compression ratio that ignores the inherent complexity of the underlying data. The Byte Latent Transformer (BLT), introduced by Meta FAIR (Pagnoni et al., 2024), addresses this by using an entropy-based patcher. A lightweight local encoder calculates byte-level entropy, and when the uncertainty spikes above a learned threshold, a new patch boundary is formed. The heavy Latent Global Transformer then processes these variable-length patches.

While the original authors demonstrated the *efficiency* of this approach (scaling up to 8B parameters), the *interpretability* of these dynamic bound-

aries remains underexplored. In natural language, entropy is widely distributed. In code, syntax keywords (e.g., `return`, `def`, `std::cout`) are highly predictable, while mathematical logic blocks are highly unpredictable. We hypothesize that BLT naturally learns to compress boilerplate code into massive, low-compute patches while isolating complex syntax nodes into high-compute, byte-level patches.

Beyond basic syntactic parsing, modern sequence models must survive noisy, real-world inputs and comprehend deep semantic logic. By operating at the byte level, BLT inherently bypasses the catastrophic failures often seen in subword tokenizers when faced with typos or obfuscation. Project ESCAPE analyzes not just what BLT parses perfectly, but how it degrades under noise and how it handles systems-level semantic complexity.

2 Research Objectives

- Syntactic Alignment:** Do entropy-triggered patch boundaries of BLT align with Abstract Syntax Tree (AST) node boundaries in C++ and Python?
- Compute Allocation Ratio:** How does the average Bytes-Per-Patch (BPP) ratio differ when processing rigid code syntax versus natural language (e.g., English Wikipedia)?
- Cross-Language Structural Recognition:** Does the model adapt its patching strategy differently for indentation-based syntax (Python) versus bracket-based syntax (C++)?
- Robustness to Code Noise:** Does the entropy-guided patching mechanism maintain its structural boundaries under character-level noise (e.g., syntax errors, obfuscation), or does it gracefully degrade without systemic failure?

5. **Semantic Compute Allocation:** Does BLT allocate higher compute density (smaller, focused patches) to memory-unsafe or semantically complex C++ operations (e.g., manual pointer manipulation) compared to standard mathematical logic?

3 Methodology

Model and Data. We use the open-source `BltForCausalLM` integrated into the Hugging Face `transformers` library, which follows a Local Encoder $\rightarrow$ Latent Transformer $\rightarrow$ Local Decoder architecture. We sample natural language from WikiText-103 and code from CodeSearchNet, filtered for Python and C++.

Latent Extraction. Since BLT resolves patches dynamically during the forward pass based on an entropy threshold ($\tau = 1.34$ by default), we implement PyTorch forward hooks on the `BltPatcher` module to extract the exact byte indices where a patch split is initiated, mapped back to the raw UTF-8 string.

AST Ground Truth. We parse code using `tree-sitter` to obtain byte-level start/end indices for structural nodes (function definitions, loops, variable declarations), and separately query pointer dereferencing (`*`), address referencing (`&`), and manual memory management (`new/delete`) as memory-unsafe regions.

Adversarial Data. We construct a noised variant of the CodeSearchNet subset by injecting missing semicolons, localized indentation changes, camelCase $\leftrightarrow$ snake_case inversions, and typos in syntax keywords, to probe robustness (Objective 4).

4 Evaluation Metrics

- **AST Boundary IoU:** Overlap between BLT patch byte-spans and `tree-sitter` AST node byte-spans; high IoU indicates organic structural understanding.
- **Compute Density (BPP):** Bytes-per-patch profiling; we expect high variance for code (large patches for keywords, tiny patches for variables) versus uniform BPP for natural language.
- **Entropy Heatmaps:** Visualization overlaying entropy scores on code to show where

the Latent Transformer allocates maximum FLOPs.

- **Degradation Shift:** Delta in IoU and BPP between clean and adversarial datasets, indicating robustness of entropy thresholds relative to subword tokenizers.
- **Memory-Specific Compute Profiling:** BPP restricted to memory-unsafe regions, testing whether BLT recognizes semantic complexity beyond boilerplate structure.

5 Anticipated Challenges

The primary challenge is managing BLT’s dual-model architecture: mapping dynamic latent states from the global transformer back to local byte sequences requires careful index tracking. Fairly comparing structural alignment across Python and C++ requires normalizing for AST depth. Reliably isolating memory-unsafe regions via `tree-sitter` requires precise query strings capturing semantic intent rather than surface syntax, and adversarial noise must be tuned to break token patterns without destroying logical validity.

6 Deliverables (Interim)

1. A functional PyTorch extraction pipeline using the Hugging Face BLT implementation that logs patch boundaries.
2. An initial AST mapping engine using `tree-sitter`, configured for general structural nodes and memory-operation nodes.
3. Preliminary BPP statistics comparing a small subset of WikiText to clean and adversarial Python code.

References

Artidoro Pagnoni, Ram Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srinivasan Iyer. 2024. Byte latent transformer: Patches scale better than tokens. *arXiv preprint arXiv:2412.09871*.